

RoboChess Arm: Teaching a Robotic Arm to Play Chess

Nazar Mykolaichuk, Marko-Zenon Peleshko, Yarema Mychasyak, and Yaropolk Kalmuk
Ukrainian Catholic University

Lviv, Ukraine

{mykolaichuk.pn, peleshko.pn, mychasyak.pn, kalmuk.pn}@ucu.edu.ua

Mentor: Maksym Shamrai
Ukrainian Catholic University
Lviv, Ukraine
maksym.shamrai@ucu.edu.ua

Abstract—This project aims to develop a robotic arm capable of recognizing and manipulating chess pieces on a real chessboard. The long-term goal is to enable the system to play chess autonomously by combining computer vision, robotic control, and game logic. The initial phase focuses on basic object detection and piece movement.

Index Terms—robotics, computer vision, automation, artificial intelligence, chess

I. INTRODUCTION

The goal of our project, *RoboChess Arm*, is to create a robotic arm that can visually recognize chess pieces and make moves according to chess rules. This project lies at the intersection of robotics, computer vision, and game logic, offering both a fun and technically challenging application of artificial intelligence and automation.

II. MOTIVATION

We wanted to explore how AI and physical systems can interact in a real-world task. Playing chess involves strategy, precision, and recognition—skills that are not only human but can also be modeled through robotics and computer vision. By teaching a robot to play chess, we can explore fundamental challenges in motion planning, perception, and intelligent decision-making.

III. SYSTEM OVERVIEW

The system consists of these key components:

- **Camera:** Captures the chessboard image and sends it to the processing unit.
- **Vision module:** Uses OpenCV to detect and identify the squares vs lues (0 - free, 1 - occupied).
- **Control unit:** Translates chess moves into physical motion commands.
- **Chess engine:** Calculates the best move in given position. The chess engine we used is called Stockfish.
- **Robotic arm:** Executes movements to pick and place pieces.

This project is for the UCU Robotics Project Presentation, May 2025, Lviv, Ukraine.

IV. TECHNOLOGIES USED

- **Python & OpenCV** — image recognition and position tracking.
- **Gazebo** — an environment for testing our scripts
- **ROS** — controlling robotic motion.
- **Python-Chess / Stockfish** — move generation and logic.
- **Camera module & servo-based arm** — physical interaction.

V. DEVELOPMENT STAGES

The project was developed in several phases:

- 1) Initialize robotic arm and basic movement control.
- 2) Implement piece grabbing and placement.
- 3) Develop vision recognition of board and squares.
- 4) Create a model that will tell if a square is occupied by some piece.
- 5) Integrate the logic engine for move calculation.
- 6) Demonstrate a full robot-vs-human game.

VI. CURRENT PROGRESS, MAIN ALGORITHM

At the start, the camera detects the board and divides it into 64 squares. Each square is then classified - either 0 (free) or 1 (occupied). Then, after opponent did their move, we look through each square and look for those that changed their values. If it was 1 and became 0 - that means a piece that was here moved. If a square changed from 0 to 1 - we found a final destination for that piece. So, we get a new position and ready to send it to another node. After that the position is sent to chess engine node. This node tells the best move for a given position. Then, this move is sent to another node - robot node. Here, we get our move that we want to be performed by xarm5 robot. Before that we manually measured each joint characteristic needed to move robochess arm from point a to point b.

VII. CAMERA NODE

Camera node every 30 seconds gets the photo from camera, this node looks for the chessboard without brown frame on the phone, divides it into 64 photos of every cell and send them to classifier.

VIII. CLASSIFIER

Classifier determines every cell if it has a figure on it or not. It watch the histogram of the cell. If histogram is the same on the every pixel of the cell - the cell is empty. However, it would be quite difficult to find the figure if its color and the color of the cell are the same. So we attached blue paper on the top of every figure. If on histogram of the color of the cell classifier sees the blue one - this cell has a figure. Then there are two options. If nothing changed, classifier says that and wait for the next photo. If something changed, classifier understands that some figure was moved. This information is sent to chess engine node. It searches the best move and send it to next node for movement.

IX. MOVEMENT

If this node gets the information from which cell to which robot should move the figure - this node performs the movement. It contains the coordinates of the robot above each of every 64 cells. Firstly, robot moves to the cell on which there is a figure to move. It takes it by the gripper, moves to the proper cell and opens the gripper to put the figure. After this movement robot goes to its start position above the board that camera on the robot could record the full board

X. CHALLENGES

Some of the main technical challenges include:

- Synchronization between visual input and motion output.
- Maintaining accuracy of the arm during repeated movements.
- Reliable detection of similar-looking chess pieces.
- Avoiding collisions on the chessboard.

XI. PROJECT REPOSITORY

The source code, hardware-related scripts, and documentation for the *RoboChess Arm* project are publicly available on GitHub:

https://github.com/Marko-Zenon/Pok_robohand